VHDL 과 Logisim-evolution 을 이용한 순차논리회로 설계 및 시뮬레이션

Sequential Logic Circuit Design and Simulation
Using VHDL and Logisim-evolution

VHDL Assignment-2

Course 디지털논리회로설계 (Digital Logic Circuit Design)

Name 남광민 (Nam, GwangMin)

Student ID 2023048295

Date November 13, 2026

목적 / Objective

같은 입력을 주어도 이전에 저장한 값과 클록을 바꾼 시점에 따라 순차논리회로의 출력은 달라진다. 본 실습에서는 두 종류의 래치와 세 종류의 플립플롭을 VHDL 로 기술하고, Logisim-evolution 에서 입력 핀을 조작하여 출력 Q 를 관찰하였다.

실습 구성 / Experiment Overview

Problem 1. RS Latch

S 와 R 로 1 비트를 설정·초기화하고 두 입력을 해제한 뒤의 출력을 살펴본다.

Problem 2. D Latch

Enable 을 열고 닫으며 데이터가 전달되는 구간과 저장되는 구간을 비교한다.

Problem 3. D Flip-Flop

두 래치에 반대 위상의 클록을 인가하고 상승 에지에서의 데이터 전달을 살펴본다.

Problem 4. JK Flip-Flop

유지·초기화·설정·반전을 구현하고 J 와 K 를 모두 1 로 둔 게이트 회로의 발진을 분석한다.

Problem 5. D Flip-Flop with Asynchronous Set and Clear

비동기 Set 과 Clear 를 추가하고 두 제어 입력이 겹칠 때의 우선순위를 확인한다.

각 문제는 VHDL 과 직접 구성한 게이트 회로를 모두 요구한다. 3 번은 VHDL 래치를 재사용하였고, 4 번 게이트 회로는 반전 조건에서 발진하였다. 5 번은 VHDL 로만 구현하였다.

Problem 1. RS Latch

목적 및 동작 원리 / Objective & Operation

S 와 R 은 각각 설정과 초기화를 지시한다. S=1, R=0 이면 Q 에 1 을 쓰고, S=0, R=1 이면 0 을 쓴다. 두 입력이 모두 0 이면 process 가 새 값을 할당하지 않아 직전 Q 를 유지한다. 두 입력이 모두 1 이면 코드에서 미정값 X 를 할당한다.

VHDL Code

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;
ENTITY RS_Latch IS
PORT (
    s : IN std_logic;
    r : IN std_logic;
    q : OUT std_logic
);
END RS_Latch;
ARCHITECTURE bhv OF RS_Latch IS
BEGIN
    process(s, r)
begin
    if s = '1' and r = '0' then
        q <= '1';
    elsif s = '0' and r = '1' then
        q <= '0';
    elsif s = '1' and r = '1' then
        q <= 'X';
    end if;
end process;
END bhv;
```

코드 1-1 RS_Latch.vhd 원문에서 빈 줄과 주석을 생략하였다.

Problem 1. RS Latch

회로 및 시뮬레이션 / Circuit & Simulation

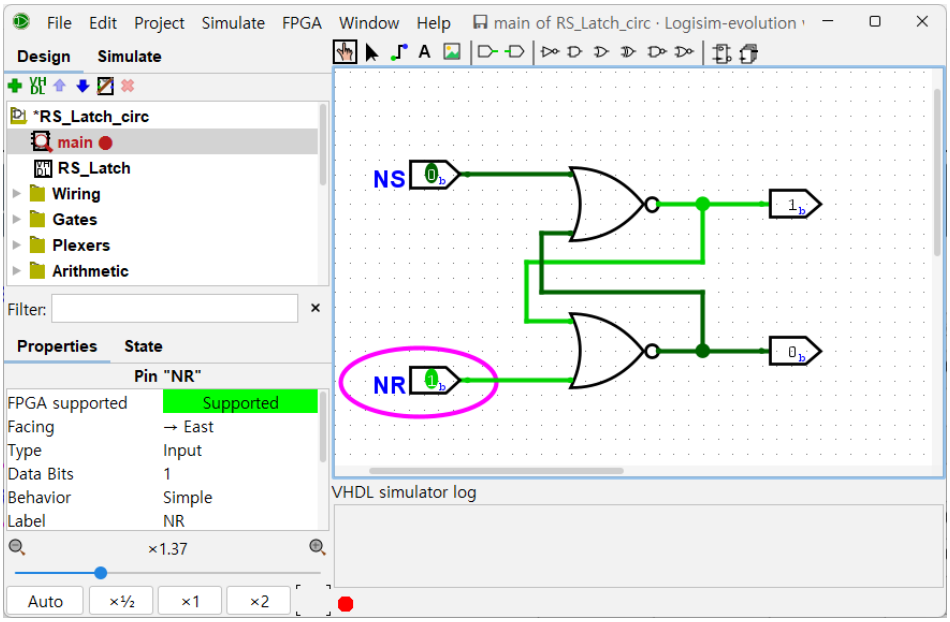


사진 1-1 교차 결합한 NOR 게이트와 두 출력 (13:35:21)

각 NOR 게이트의 출력이 다른 NOR 게이트의 입력으로 되돌아간다. NS·NR 은 그림에 표시된 핀 이름이다. 두 외부 입력이 0 일 때는 피드백으로 서로 반대인 두 출력이 유지된다.

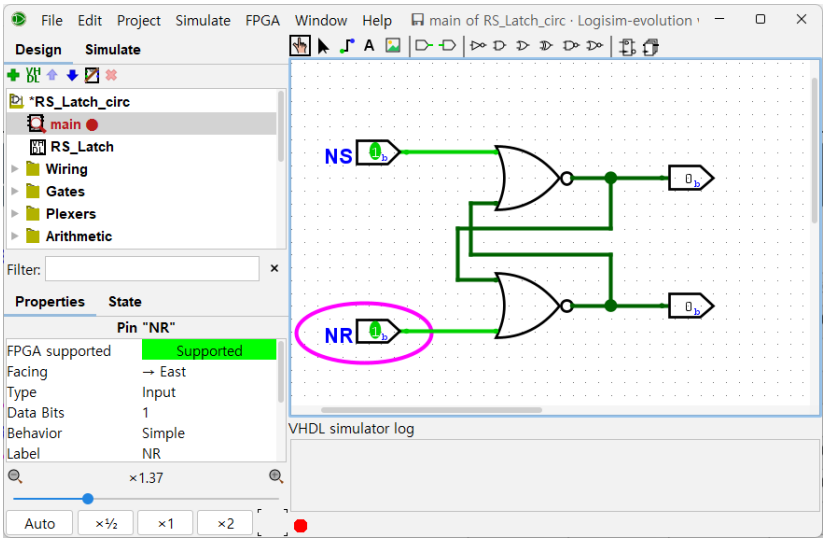


사진 1-2 두 NOR 입력을 모두 1 로 둔 금지 상태 (13:35:34)

두 입력을 모두 1 로 두면 두 NOR 출력이 동시에 0 이 되어 상보 관계가 깨진다. 이 상태에서 두 입력을 동시에 해제하면 어느 값이 저장될지 보장할 수 없다.

Problem 1. RS Latch

시뮬레이션 결과 및 고찰 / Results & Discussion

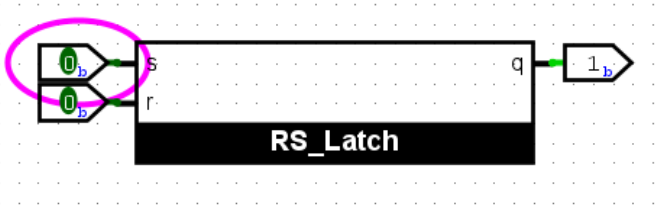


사진 1-3 s=0, r=0 에서 q=1 이 남아 있는 상태 (13:39:45)

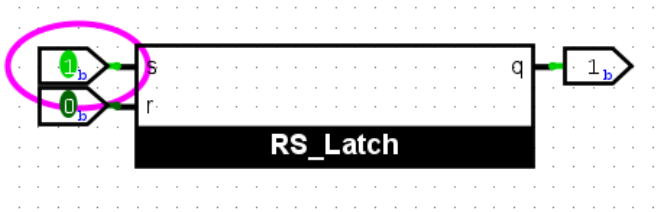


사진 1-4 s=1, r=0 에서 q=1 (13:39:49)

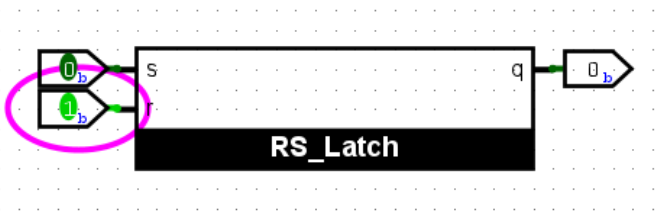


사진 1-5 s=0, r=1 에서 q=0 (13:39:53)

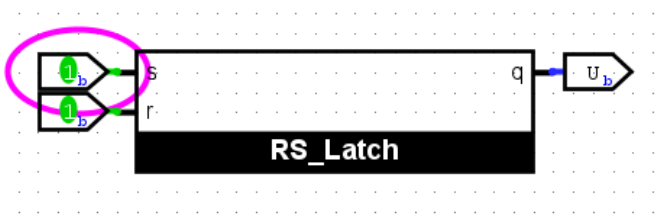


사진 1-6 s=1, r=1 에서 출력 핀에 U 표시 (13:39:57)

마지막 화면에서 Logisim 핀에 표시된 문자는 U 이고, VHDL 원문에서 할당한 값은 X 이다. 화면만으로 표시가 달라진 원인은 확인할 수 없다. 금지 입력을 해제한 뒤의 출력도 이 화면에서 결정할 수 없다.

Problem 2. D Latch

목적 및 동작 원리 / Objective & Operation

en 이 1 인 구간에는 D 의 변화가 그대로 Q 에 전달된다. en 이 0 이 되면 그 직전에 저장한 값이 남는다. process 의 감지 목록에는 d 와 en 이 모두 들어 있으므로 en 을 1 로 유지한 채 d 만 바꾸어도 할당문이 실행된다.

VHDL Code

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;
ENTITY D_Latch IS
PORT (
    d  : IN std_logic;
    en : IN std_logic;
    q  : OUT std_logic
);
END D_Latch;
ARCHITECTURE bhv OF D_Latch IS
BEGIN
    process(d, en)
begin
    if en = '1' then
        q <= d;
    end if;
end process;
END bhv;
```

코드 2-1 D_Latch.vhd 원문에서 빈 줄과 주석을 생략하였다.

Problem 2. D Latch

회로 및 시뮬레이션 / Circuit & Simulation

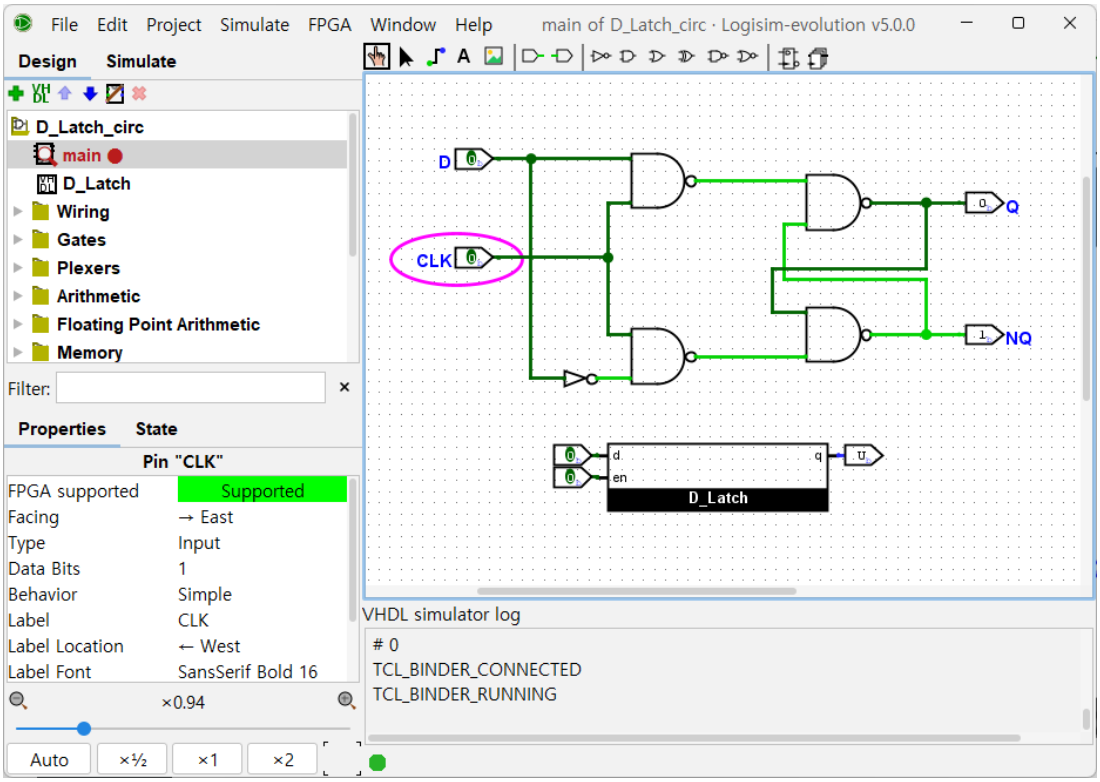


사진 2-1 NAND 네 개와 NOT 한 개로 구성한 D 래치 (13:54:43)

위쪽 회로에서 D 와 그 반전값은 각각 CLK 로 표시한 활성화 입력과 NAND 연산을 거친다. CLK=0 이면 두 입력 NAND 의 출력은 모두 1 이 되어 오른쪽의 교차 결합 NAND 가 이전 값을 유지한다. CLK=1 이면 한쪽만 0 이 되어 Q 가 D 를 따라간다.

아래쪽 VHDL 엔티티는 같은 활성화 신호를 en 으로 표시한다. 사진 2-1 에서 초기 출력은 U 이다. 코드는 초기값을 지정하지 않으므로 en 을 열어 값을 쓰기 전에는 알려진 초기 상태를 가정할 수 없다.

Problem 2. D Latch

시뮬레이션 결과 및 고찰 / Results & Discussion

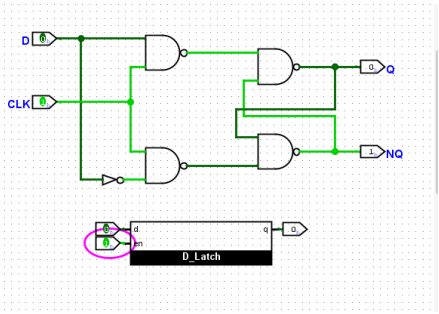


사진 2-2 d=0, en=1 에서 q=0 (13:54:57)

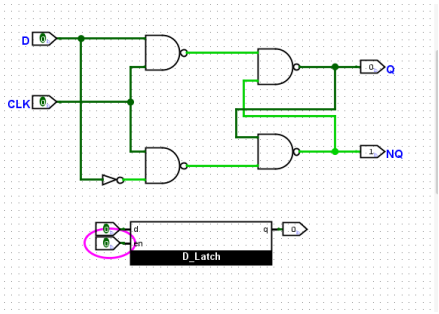


사진 2-3 en 을 0 으로 둔 상태에서 q=0 (13:55:02)

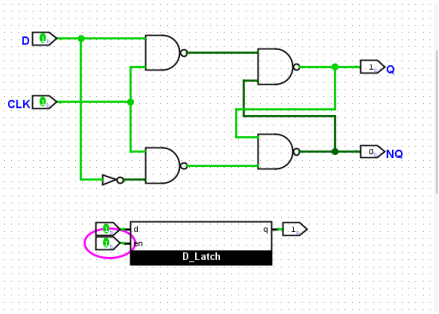


사진 2-4 d=1, en=1 에서 q=1 (13:55:15)

두 구현에서 활성화 입력이 1 인 화면의 Q 는 D 와 일치하였다. en=0 에서 D 를 여러 번 바꾸어도 Q 가 유지되는지는 이 세 화면만으로 모두 확인할 수 없으며, 유지 조건은 코드의 미할당 분기와 NAND 피드백에서 설명된다.

Problem 3. D Flip-Flop

목적 및 동작 원리 / Objective & Operation

상승 에지 직전에 입력한 D 를 저장하고 다음 상승 에지까지 Q 를 유지하는 회로이다.
VHDL 에서는 rising_edge(clk)가 참일 때만 q 에 d 를 할당하였다. 따라서 클록이 1 로 유지되는 동안 D 가 바뀌어도 Q 는 바뀌지 않는다.

VHDL Code

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;
ENTITY D_Flip_Flop IS
PORT (
    clk : IN std_logic;
    d   : IN std_logic;
    q   : OUT std_logic
);
END D_Flip_Flop;
ARCHITECTURE bhv OF D_Flip_Flop IS
BEGIN
    process(clk)
begin
    if rising_edge(clk) then
        q <= d;
    end if;
end process;
END bhv;
```

코드 3-1 D_Flip_Flop.vhd 원문에서 빈 줄과 주석을 생략하였다.

Problem 3. D Flip-Flop

회로 및 시뮬레이션 / Circuit & Simulation

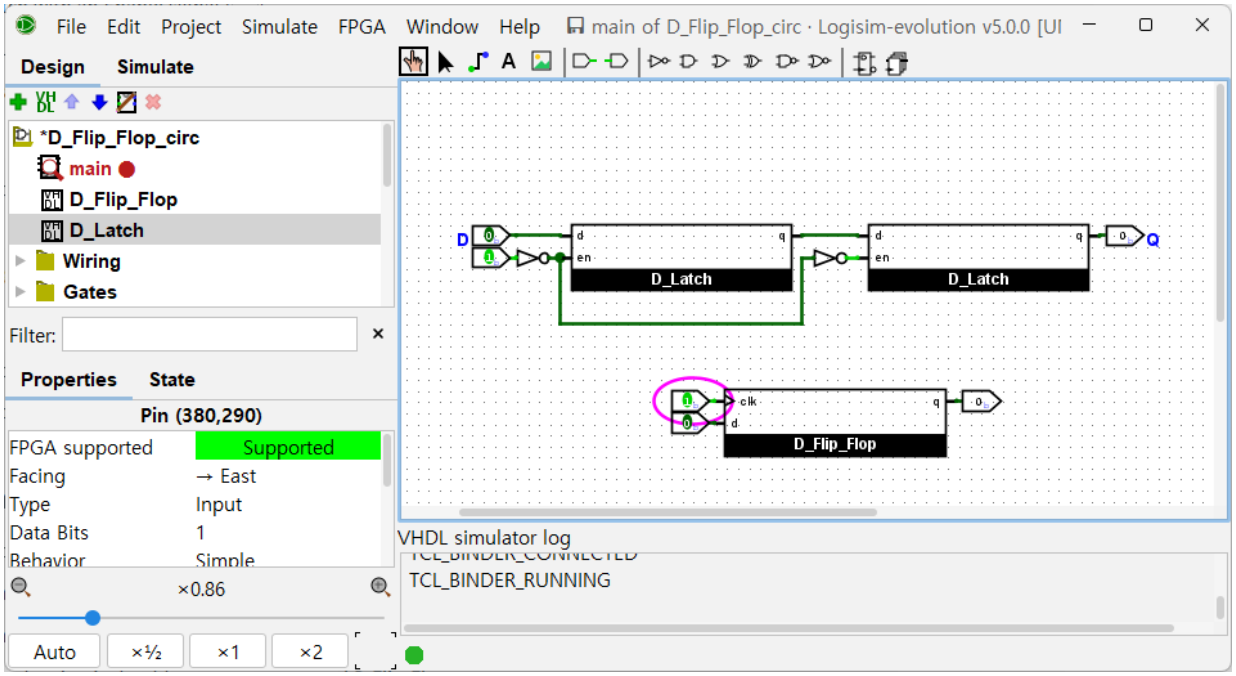


사진 3-1 위쪽의 D_Latch 두 개와 아래쪽의 D_Flip_Flop 엔티티 (14:07:11)

위쪽에서 첫 번째 래치의 Q 를 두 번째 래치의 D 에 연결하였다. 첫 번째 NOT 출력은 마스터의 en 에 들어간다. 이를 다시 반전한 신호가 슬레이브의 en 에 들어가므로 두 래치는 서로 다른 클록 구간에 열린다.

마스터와 슬레이브의 동작

CLK=0 이면 마스터가 D 를 따라가고 슬레이브는 이전 Q 를 유지한다. CLK 가 0 에서 1 로 바뀌면 마스터가 닫히고 슬레이브가 열려 마스터에 저장된 값이 최종 Q 로 전달된다.

실제 배치한 두 D_Latch 는 VHDL 엔티티이다. 두 단의 연결 및 클록 반전은 구성했으나 래치 내부를 개별 게이트로 직접 구성하지 않아, 과제가 요구한 수동 게이트 구현을 완성하지 못하였다.

Problem 3. D Flip-Flop

시뮬레이션 결과 및 고찰 / Results & Discussion

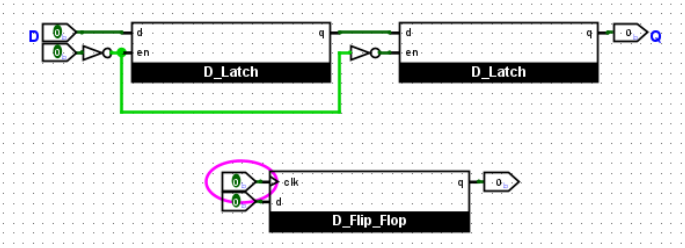


사진 3-2 CLK=0, D=0, Q=0 (14:07:20)

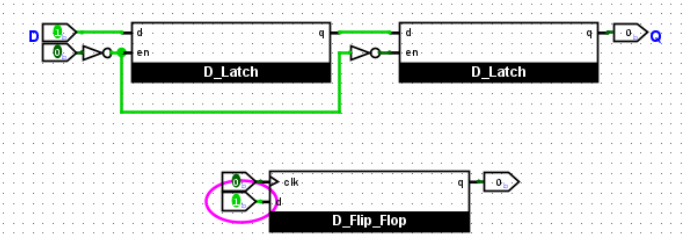


사진 3-3 CLK=0 에서 D 를 1 로 변경한 뒤에도 Q=0 (14:07:25)

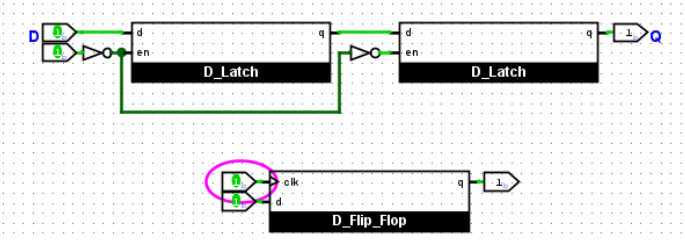


사진 3-4 CLK 를 1 로 올린 뒤 Q=1 (14:07:30)

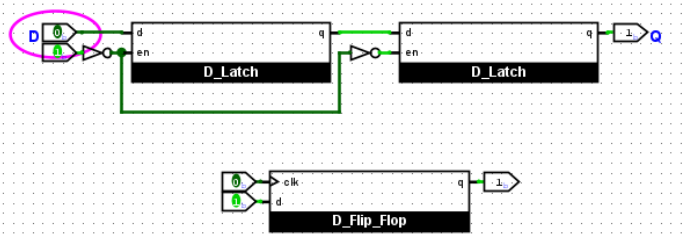


사진 3-5 위쪽 회로에서 CLK=1 인 채 D 를 0 으로 바꾼 뒤에도 Q=1 (14:07:35)

사진 3-3 과 3-4 를 비교하면 D=1 이 먼저 준비된 뒤 클록이 올라간 시점에 출력이 1 이 되었다. 사진 3-5 에서는 마스터 입력이 0 으로 바뀌었지만 최종 Q 에는 상승 에지 때의 1 이 남아 있다.

Problem 4. JK Flip-Flop

목적 및 동작 원리 / Objective & Operation

상승 에지에서 J 와 K 를 읽는다. 입력이 00 이면 현재 상태를 유지하고, 01 이면 0, 10 이면 1 을 저장한다. 11 이면 직전 상태를 한 번 반전한다. 내부 신호 q_int 는 0 으로 초기화하며 출력 q 에는 q_int 를 연결하였다.

VHDL Code

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;
ENTITY JK_Flip_Flop IS
PORT (
    clk : IN std_logic;
    j   : IN std_logic;
    k   : IN std_logic;
    q   : OUT std_logic
);
END JK_Flip_Flop;
ARCHITECTURE bhv OF JK_Flip_Flop IS
signal q_int : std_logic := '0';
BEGIN
    process(clk)
begin
    if rising_edge(clk) then
        if j = '0' and k = '0' then
            q_int <= q_int;
        elsif j = '0' and k = '1' then
            q_int <= '0';
        elsif j = '1' and k = '0' then
            q_int <= '1';
        elsif j = '1' and k = '1' then
            q_int <= not q_int;
        end if;
    end if;
end process;
q <= q_int;
END bhv;
```

코드 4-1 JK_Flip_Flop.vhd 원문에서 빈 줄과 주석을 생략하였다.

Problem 4. JK Flip-Flop

회로 및 시뮬레이션 / Circuit & Simulation

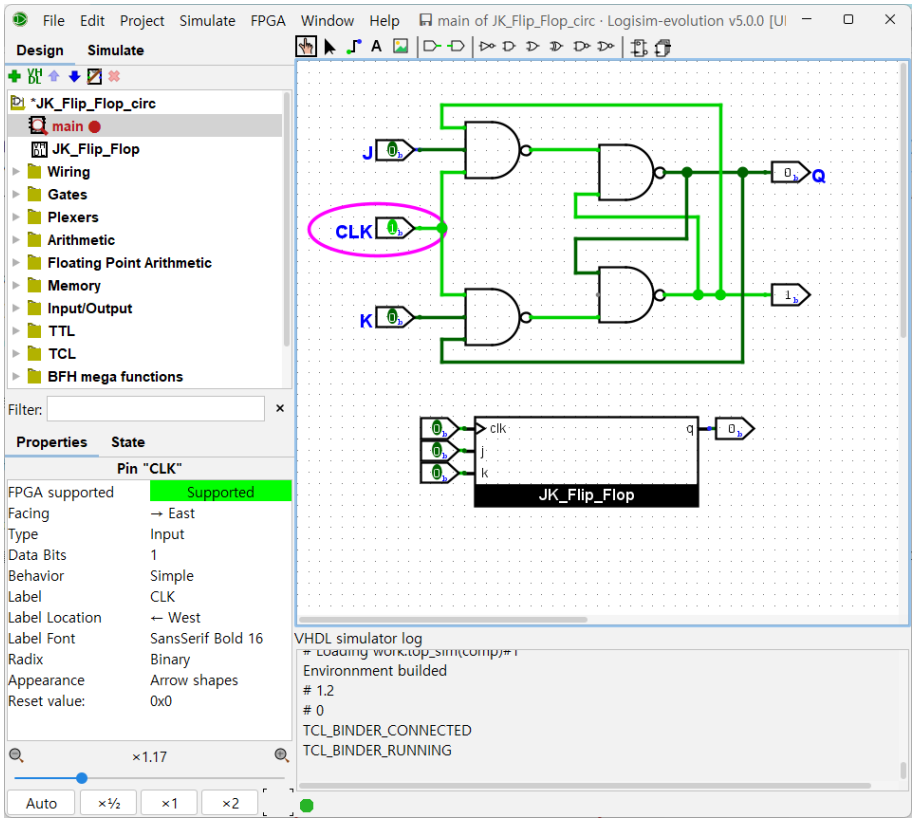


사진 4-1 게이트로 구성한 JK 회로와 별도의 VHDL 엔티티 (14:21:43)

위쪽에서는 J, K, CLK 와 두 출력의 피드백을 입력단 NAND 에 연결하였다. CLK 가 1 인 동안 입력단이 계속 열려 있어 저장부의 출력이 바뀌면 그 값이 다시 입력단에 영향을 준다. 아래쪽 엔티티에서는 상승 에지를 검사한 뒤에만 q_int 를 갱신한다.

반전 조건에서 예상되는 차이

직전 Q 가 0 이고 J=K=1 이면 정상적인 JK 플립플롭은 첫 상승 에지에서 1, 다음 상승 에지에서 0 을 저장한다. 두 에지 사이에 CLK 가 계속 1 인 동안에는 추가 반전이 없어야 한다.

Problem 4. JK Flip-Flop

시뮬레이션 결과 및 고찰 / Results & Discussion

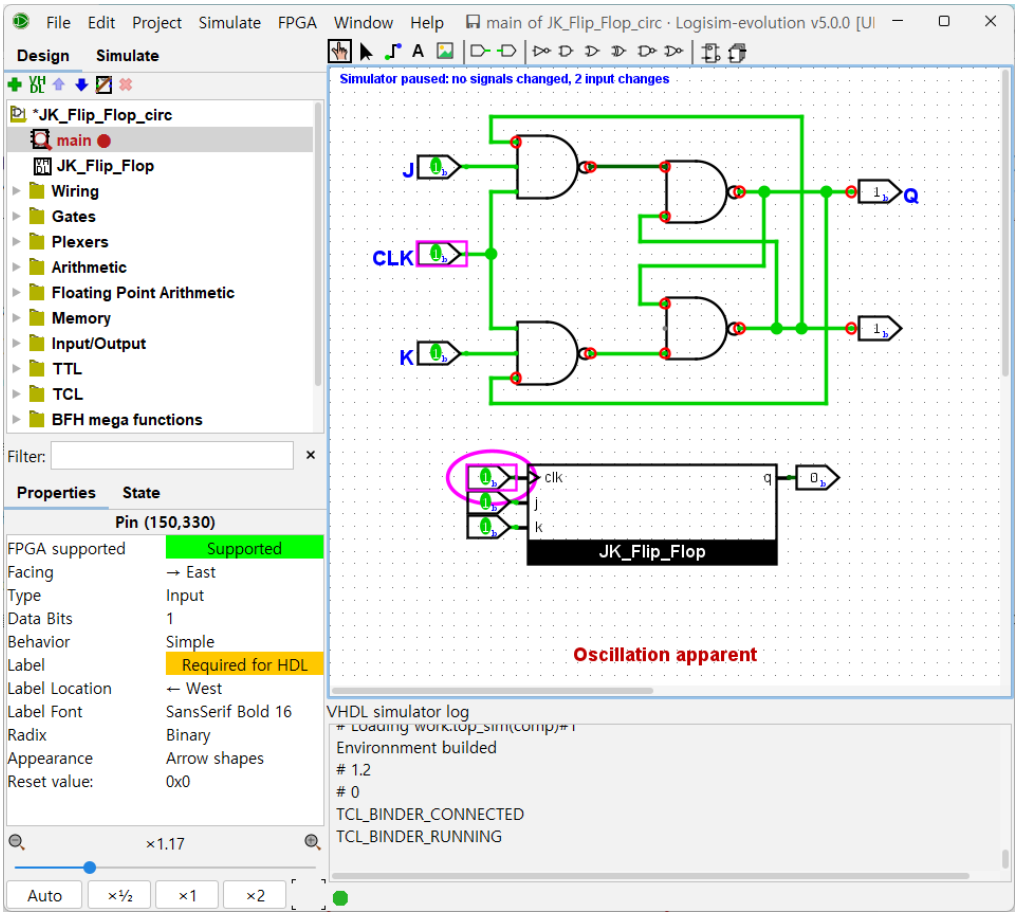


사진 4-2 J=K=CLK=1 에서 발생한 Oscillation apparent 경고 (14:23:41)

Logisim 이 위쪽 게이트 경로를 빨간 점으로 표시하고 발진 경고를 출력하였다. 이때 표시된 출력은 안정된 최종값으로 읽을 수 없다. J=K=1 인 동안 한 번 바뀐 Q 가 다시 입력에 반영되어 같은 클록 구간 안에서 반전이 반복되는 레이스 어라운드(race-around) 현상이다.

따라서 위쪽 게이트 회로는 유효한 상승 에지 트리거 JK 플립플롭이 아니다. 아래쪽 VHDL 은 상승 에지마다 한 번만 반전하도록 작성하였다. 사진 4-2 에서 아래쪽 q 한 값만으로 연속한 두 에지의 반전까지 검증할 수는 없다.

Problem 5. D Flip-Flop with Asynchronous Set and Clear

목적 및 동작 원리 / Objective & Operation

이 회로는 클록과 관계없이 상태를 바꿀 수 있는 두 입력을 갖는다. clr 가 1 이면 Q 는 0 이 되고, clr 가 0 이며 set 이 1 이면 Q 는 1 이 된다. 둘 다 0 일 때만 상승 에지에서 D 를 읽는다. 코드에서 clr 를 먼저 검사하므로 set 과 clr 가 동시에 1 이어도 Q 는 0 이다.

VHDL Code

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;
ENTITY D_Flip_Flop_Asy IS
PORT (
    clk : IN std_logic;
    d   : IN std_logic;
    set : IN std_logic;
    clr : IN std_logic;
    q   : OUT std_logic
);
END D_Flip_Flop_Asy;
ARCHITECTURE bhv OF D_Flip_Flop_Asy IS
BEGIN
    process(clk, set, clr)
begin
    if clr = '1' then
        q <= '0';
    elsif set = '1' then
        q <= '1';
    elsif rising_edge(clk) then
        q <= d;
    end if;
end process;
END bhv;
```

코드 5-1 D_Flip_Flop_Asy.vhd 원문에서 빈 줄과 주석을 생략하였다.

Problem 5. D Flip-Flop with Asynchronous Set and Clear

회로 및 시뮬레이션 / Circuit & Simulation

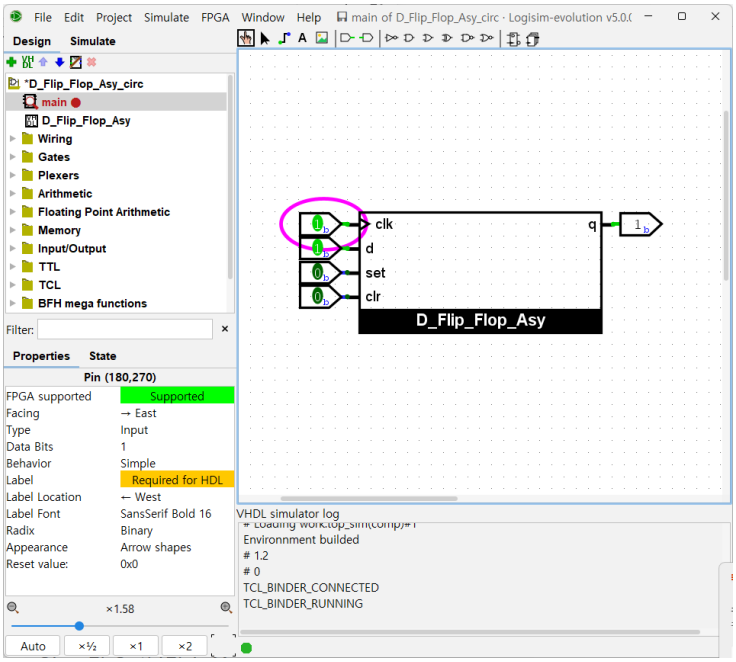


사진 5-1 네 입력 핀과 D_Flip_Flop_Asy 엔티티 (14:43:24)

이 문제의 Logisim 파일에는 VHDL 엔티티와 입출력 핀만 있다. 비동기 제어가 있는 D 플립플롭은 VHDL 로만 구현하였으며, 문제마다 별도로 요구된 게이트 수준 구현 요건을 충족하지 못하였다.

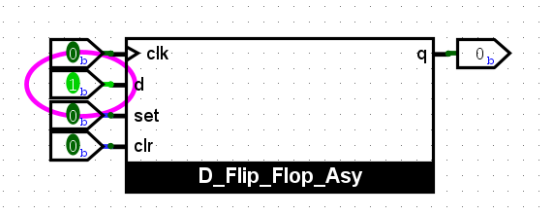


사진 5-2 CLK=0, D=1, Q=0 (14:43:18)

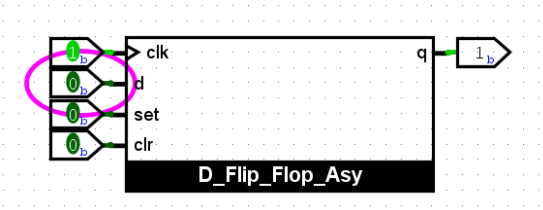


사진 5-3 CLK=1 에서 D 를 0 으로 바꾼 뒤에도 Q=1 (14:43:46)

시간순으로 사진 5-2, 5-1, 5-3 을 보면 제어 입력을 모두 0 으로 둔 상태에서 상승 에지에 D=1 이 저장되고 이후 D=0 이 되어도 Q=1 을 유지하였다.

Problem 5. D Flip-Flop with Asynchronous Set and Clear

비동기 입력 실험 및 고찰 / Results & Discussion

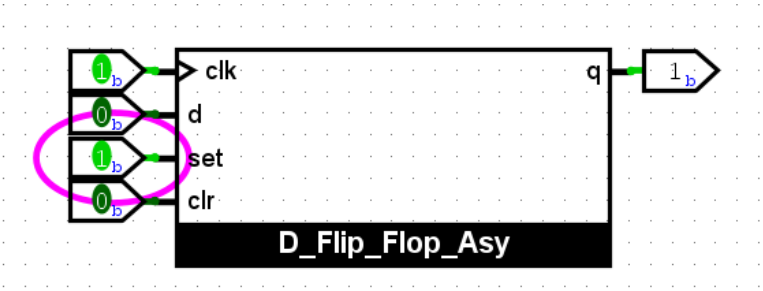


사진 5-4 set=1, clr=0 에서 Q=1 (14:44:02)

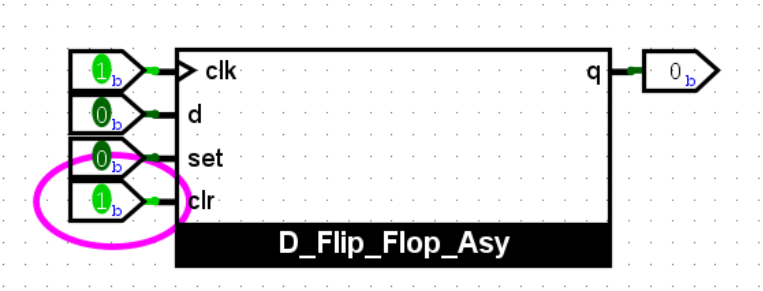


사진 5-5 set=0, clr=1 에서 Q=0 (14:44:12)

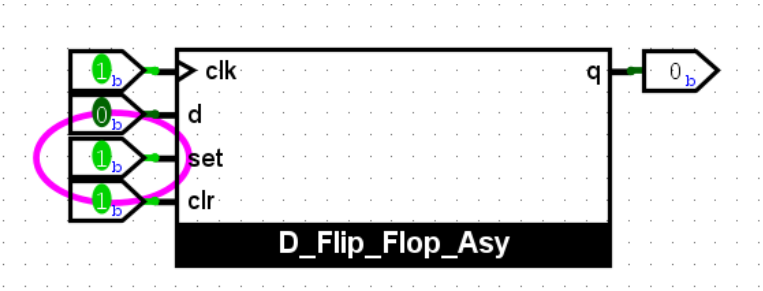


사진 5-6 set=1, clr=1 에서 Q=0 (14:44:19)

세 화면 모두 CLK=1, D=0 이다. Clear 단독 및 동시 활성 상태에서 Q=0 이므로 코드의 Clear 우선순위와 일치한다. Set 단독 화면을 찍기 직전에도 Q=1 이었다. 따라서 그 장면은 Set 활성 상태의 출력은 보여 주지만 Set 만으로 0 에서 1 로 변하는 순간까지 보여 주지는 않는다.

결론 / Conclusion

RS 래치는 두 제어 입력을 해제해도 피드백으로 상태를 보존한다. D 래치는 en 이 열린 구간 동안 D 를 받아들이며, D 플립플롭은 상승 에지에서 읽은 값을 다음 에지까지 유지한다. 사진 3-3~3-5 에서 D 의 변화와 Q 의 변화가 서로 다른 시점에 일어나는 모습을 확인하였다.

3 번의 마스터-슬레이브 연결은 반대 위상의 두 래치로 에지 동작을 설명하지만, 사용한 래치는 VHDL 엔티티였다. 4 번에서는 피드백 경로가 CLK=1 인 동안 계속 열려 있어서 J=K=1 일 때 발진하였다. 두 경우 모두 VHDL 코드의 동작과 게이트 구현의 완성도는 별도로 판단해야 한다.

5 번에서는 Clear, Set, 상승 에지 순서로 조건을 검사하였다. 두 제어 입력을 동시에 1 로 둔 사진 5-6 에서 Q 는 0 이었다. 이 결과는 VHDL 구현에 해당하며 별도의 게이트 회로는 구현하지 않았다.